Politecnico di Torino
Master's Degree in Data Science and Engineering

Langevin Particle Swarm Optimization with Friction-Based Communication

Supervisors:
Prof. Luigi Preziosi
Prof. Marco Scianna

Candidate:
Ivan Ludvig Tereshko

Turin, 2025

Optimization Problem

Find input value that **minimizes** a given function: $\min_{\mathbf{x} \in \mathbb{R}^d} U(\mathbf{x})$

Possible applications:

- Tuning parameters in machine learning models
- Finding shortest transportation routes; reducing production costs

Swarm-Based Optimization methods:

- **Particle Swarm Optimization (PSO):** agents are influenced by **individual** and **social** components
- **Swarm-Based Gradient Descent (SBGD):** agents exchange “mass”, determining their step sizes along the negative gradient

Langevin Dynamics

A particle suspended in a viscous fluid obeys the **Langevin equation**:

$$m\ddot{x} = -\mu\dot{x} + F(t) + \omega(t)$$

m – mass, *μ* – friction coefficient, *F* – force with potential *U*

ω – **random noise** from collisions: $\langle\omega(t)\rangle = 0, \quad \langle\omega(t)\omega(t')\rangle = 2\mu T\delta(t-t')$

Overdamped case: $m\ddot{x} \approx 0$

$$\dot{x} = -\frac{1}{\mu}\frac{\partial U}{\partial x} + \eta(t)$$

Diffusion coefficient: $D = T/\mu$, noise term: $\langle\eta(t)\eta(t')\rangle = 2D\delta(t-t')$

Resulting dynamics: **Brownian motion**

Mathematical Model

Each particle *i* is characterized by its **position** $\boldsymbol{x}_i \in \mathbb{R}^d$ and **friction** coefficient μ_i, obeying:

$$\dot{\boldsymbol{x}}_i = -\frac{1}{\mu_i}\left(\nabla U(\boldsymbol{x}_i) + \boldsymbol{\omega}_i(t)\right)$$

$$\dot{\mu}_i = -f(\mu_i)\sum_{j\in\mathcal{A}} \frac{U(\boldsymbol{x}_i) - U(\boldsymbol{x}_j)}{U_{\max} - U_{\min}} \qquad i \in \mathcal{A}$$

N – number of particles, $\delta\mu$ – minimum friction coefficient threshold

Active particles set: $\mathcal{A} = \{i \in \{1, \ldots, N\} \mid \mu_i > \delta\mu\}$

Friction response rate function: $f(\mu) = \dfrac{\sqrt{\mu}}{1+\sqrt{\mu}}$

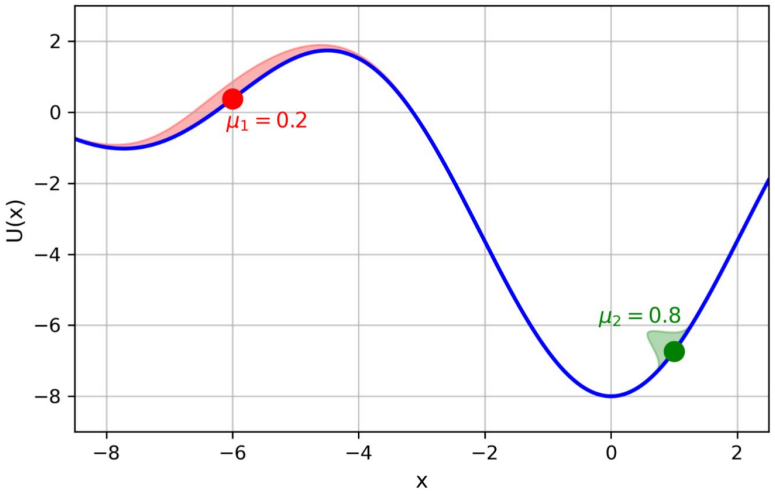
LPSF: **L**angevin **P**article **S**warm Optimization with **F**riction-Based Communication

Gaussian Random Force

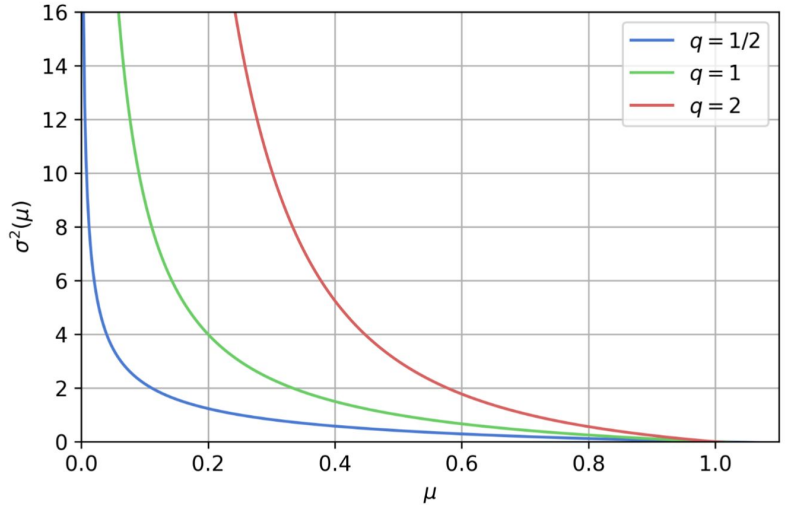
Small friction coefficient: bigger noise variance → better **exploration**

Large friction coefficient: small noise variance → careful **exploitation**

$$\sigma^2(\mu) = \begin{cases} \frac{1}{\mu^q} - 1, & \mu \leq 1 \\ 0, & \text{otherwise} \end{cases}$$



Variances of noise forces

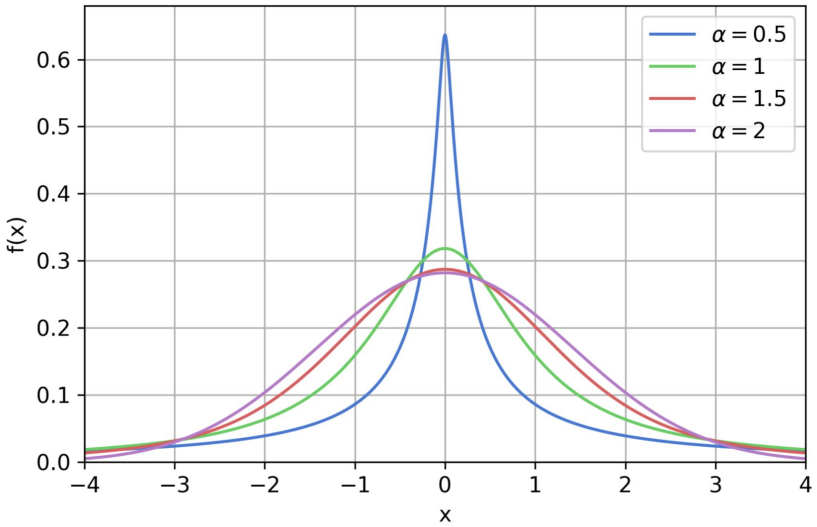


Variance functions

Levy-stable Distribution

Levy-stable distribution: $\varphi(k) = \exp\left(-c^{\alpha}|k|^{\alpha}\right)$

- **Heavy-tailed** stable distribution
- Stability index: $\alpha \in (0, 2]$
- Scale factor *c*
- Cauchy distribution: $\alpha = 1$
- Gaussian distribution: $\alpha = 2$

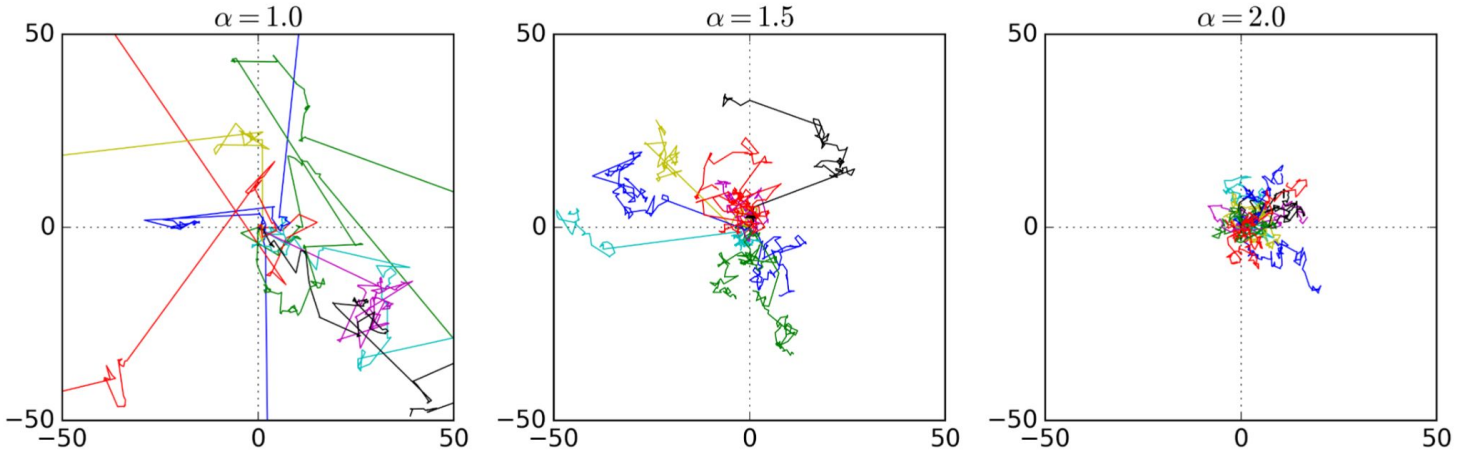


Probability density functions (c=1)

Levy Flights

Random walk with steps in random directions with **Levy-stable distributed lengths**

- Fractal-like structure
- Series of small steps interrupted by large jumps

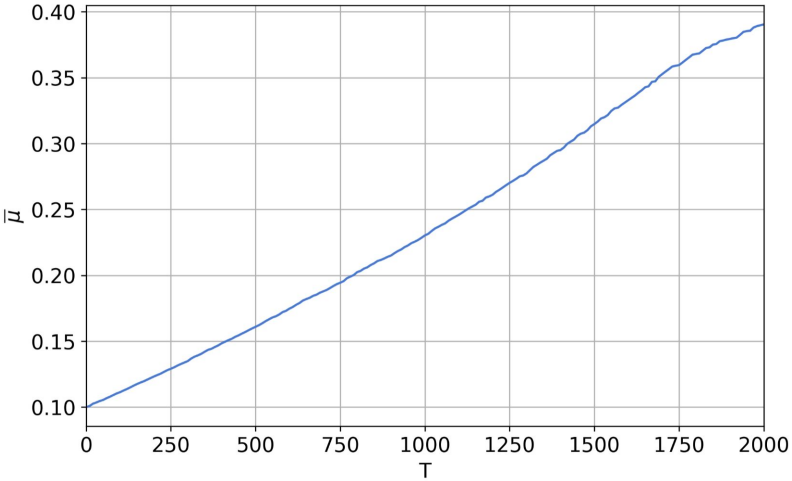


Levy flights for different stability indices

Emergent Annealing

Conserved quantity: $M = \sum_{i \in \mathcal{A}} (\mu_i + 2\sqrt{\mu_i})$

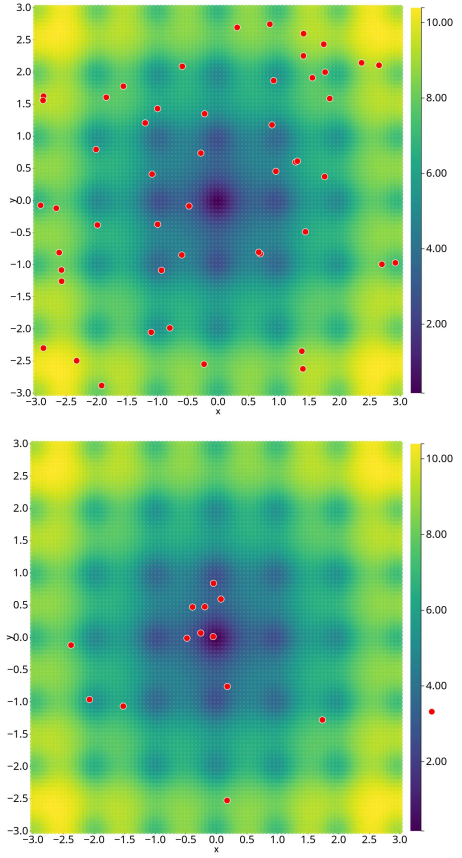
- Average friction coefficient $\bar{\mu}$ increases over time → particles **slow down** → swarm **settles**
- $1/\bar{\mu}$ is analogous to **temperature** in **Simulated Annealing**



Average friction coefficient in an experiment (N=500, Gaussian noise)

Implementation

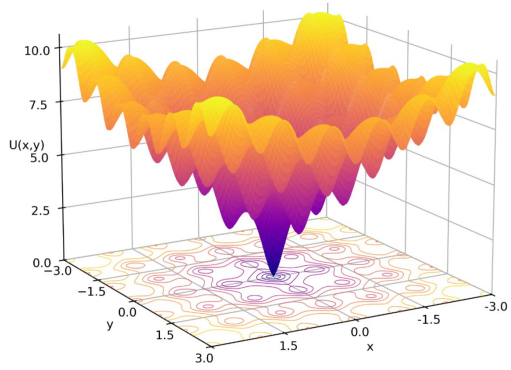
- Solve using **Euler's method** for *T* timesteps
- Track **best obtained solution**
- **Early stopping** if no improvement after T_{early} steps
- Initialization: $\mathbf{x}_i^{(0)} \sim \mathcal{U}(\mathcal{S})$, $\mu_i^{(0)} = \mu_0$
- **GPU** implementation with *wgpu*



Simulation at different timesteps

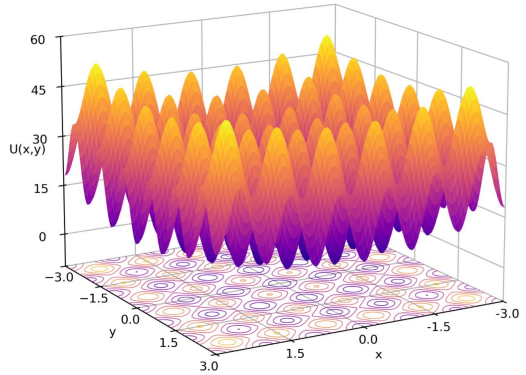
Benchmark Functions

Ackley



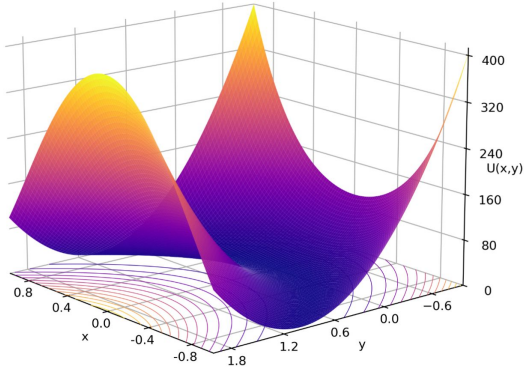
$$f(\mathbf{x}) = 20 + e - 20\exp\left(-0.2\sqrt{\frac{\sum_{i=1}^{d} {x_i}^2}{d}}\right) - \exp\left(\frac{\sum_{i=1}^{d}\cos(2\pi x_i)}{d}\right)$$

Rastrigin



$$f(\mathbf{x}) = \sum_{i=1}^{d} \left({x_i}^2 - 10\cos(2\pi x_i) + 10\right)$$

Rosenbrock



$$f(\mathbf{x}) = \sum_{i=1}^{d-1} \left(100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2\right)$$

Success criteria: $\|\boldsymbol{x}^* - \boldsymbol{x}_o\| < \delta r = 0.1$

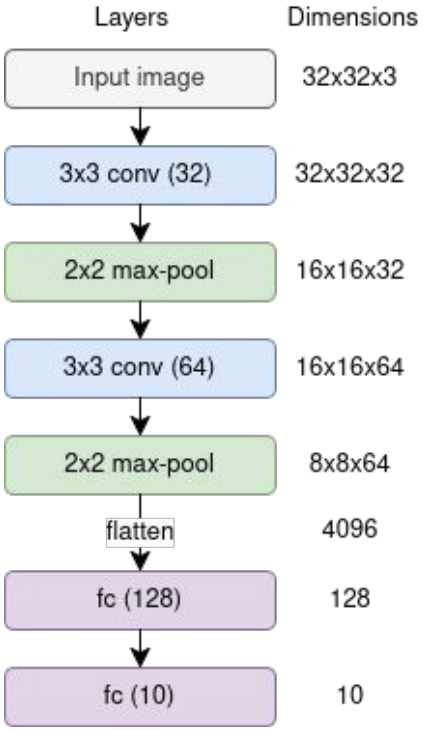
Benchmark Function Optimization Results

- **Better** performance for **bigger swarms**
- Performance **drops** when problem **dimension increases**
- **No noise type** consistently **outperforms** others across all functions
- **Shifting** initialization range **decreases** performance

Function	$N = 10$			$N = 50$			Adam
	PSO	SBGD	LPSF	PSO	SBGD	LPSF	
Rastrigin, $d = 2$	91.4%	33.5%	94.2%	100%	89.8%	100%	5%
Rastrigin, $d = 3$	62.5%	4.5%	45.2%	99.8%	25.5%	88.8%	1%
Ackley, $d = 6$	78.4%	0.9%	91.4%	100%	100%	100%	0%
Rosenbrock, $d = 3$	38.1%	2.3%	99.5%	100%	35.5%	100%	100%
Rosenbrock, $d = 6$	1.3%	0.0%	37.8%	77.3%	1.4%	61.3%	82%

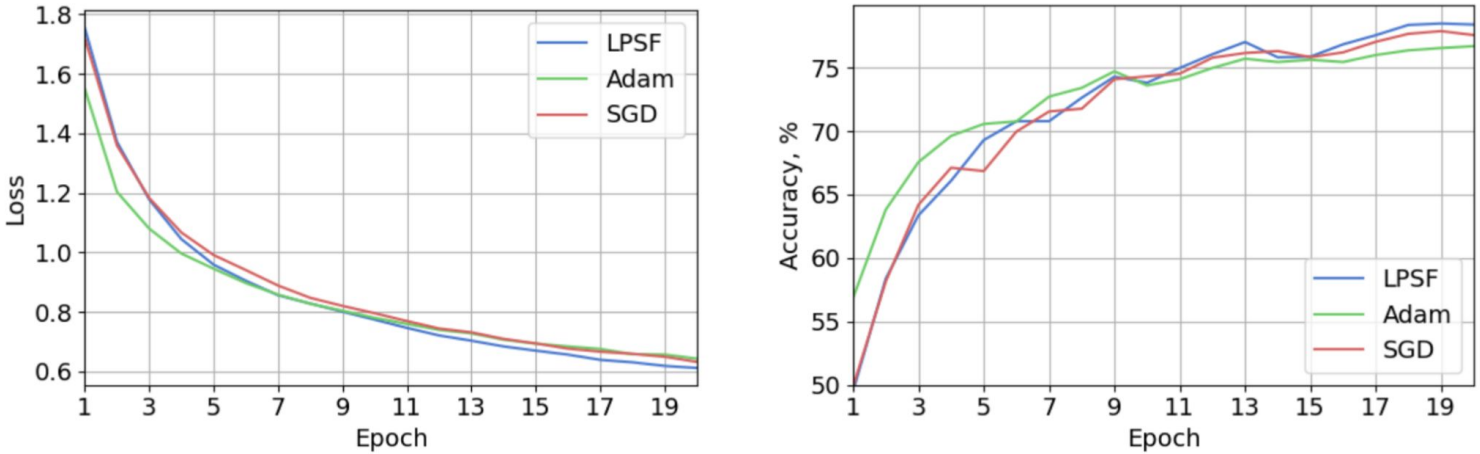
Neural Network Optimization

- **CIFAR-10** dataset
- **Preprocessing:** random cropping and horizontal flipping, colour normalization
- Simple **CNN** with two convolutional layers
- Cross-entropy loss
- Comparison with **SGD, Adam**



NN architecture

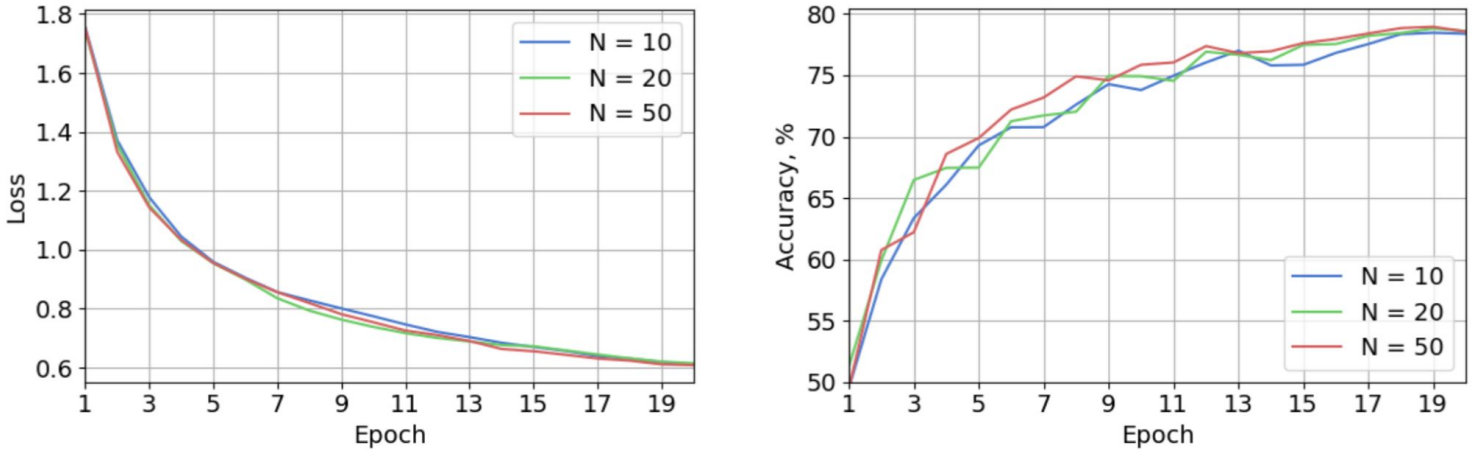
Neural Network Optimization Results



Loss and accuracy during training

Optimizer	Loss	Accuracy
LPSF	0.612	78.47%
Adam	0.643	76.69%
SGD	0.632	77.87%

Neural Network Optimization Results



Loss and accuracy during training

N	Loss	Accuracy
10	0.611	78.47%
20	0.612	78.82%
50	0.608	78.96%

Conclusions

- Communication enables **exploration** and **exploitation**
- **Emergent annealing** settles the swarm
- **Strong** performance on **benchmark functions**
- Slight improvement in NN optimization, but significantly **higher computational cost**
- Algorithm is sensitive to **hyperparameters**
- **Future work:** theoretical analysis and model modification